Recalibrating VLA Baselines: Conservative Finetuning Closes the Generalization Gap on LIBERO

Anonymous Author(s)
Affiliation
Address
email

Abstract: Vision-language-action models achieve strong performance on standard benchmarks but are brittle to small perturbations. On LIBERO-PRO's position-swap evaluation, the published $\pi_{0.5}$ checkpoint averages 21% success compared to 96% on standard LIBERO. We systematically investigate three hypotheses for this gap: that the training recipe is too aggressive, that LoRA better preserves generalization, and that richer visual features from video diffusion models could help. Of these, only the training recipe produces a meaningful change. A conservative full-finetuning recipe doubles position-swap performance to 42% (stable across training durations from 8 k to 27 k steps) while matching standard LIBERO scores, with no architectural modification. The effect is bounded: too-aggressive recipes memorize trajectories, while too-conservative recipes fail to learn appearance-based identification at all. LoRA and frozen video diffusion priors do not improve over the published baseline. Our results suggest that the widely-used $\pi_{0.5}$ checkpoint reflects recipe-induced brittleness rather than a fundamental representational limit. We release our configurations as a calibrated baseline for future work.

Keywords: vision-language-action models, robot learning, finetuning, generalization, perturbation benchmarks

1 Introduction

Vision-language-action models have reached impressive performance on standard manipulation benchmarks. The published $\pi_{0.5}$ [1] checkpoint averages 96% on LIBERO [2]. LIBERO-PRO [3] reveals what these scores conceal: when objects exchange positions and the policy must identify targets by appearance rather than where it had memorized them to be, the same checkpoint averages just 21% on position-swap suites. The likely failure mode is trajectory memorization. LIBERO provides 50 demonstrations per task with objects in nearly identical positions across every demonstration. A policy that memorizes the mapping between the current observation and the next action, conditioned on where objects were during training, will ace standard evaluation without ever encoding what target objects look like. Position-swap evaluation breaks this shortcut: the butter is now where the tomato sauce was, and the policy reaches for the wrong object.

Recent work treats this brittleness as an architectural limitation. AFI [4] adds 3D affordance fields and GPT-4o at inference time to guide the policy toward displaced objects. VLS [5] steers a frozen policy via VLM-based gradient signals at each denoising step. VEGA-3D [6] fuses frozen video diffusion features for richer spatial understanding. These methods introduce substantial inference-time compute to compensate for base-policy failure, and all take the published checkpoint's memorization as a given.

Conventional wisdom holds that LoRA [7] is preferable to full finetuning (FFT) for VLA adaptation, primarily because it preserves the pretrained model's general capabilities. OpenVLA [8] reports

LoRA matches FFT on in-distribution tasks at a fraction of the trainable parameters. VLM2VLA [9] argues FFT causes catastrophic forgetting of VLM capabilities. VLA-GSE [10] shows FFT degrades multimodal understanding on benchmarks like MMMU. These comparisons evaluate on in-distribution or standard generalization metrics. Whether LoRA's preservation of VLM capability translates to better robotic generalization under perturbation remains untested. We also note that OpenVLA-OFT [11] demonstrated that finetuning recipe changes alone raised OpenVLA's LIBERO average from 77% to 97%, suggesting the field may underestimate recipe effects.

We tested three hypotheses about the source of this memorization. First, the training recipe: the published $\pi_{0.5}$ recipe uses a batch size of 256 and learning rate of $5e{-}5$. With only 50 demonstrations per task, these aggressive hyperparameters appear to cause rapid convergence onto position shortcuts before visual features are encoded. Second, finetuning strategy: conventional wisdom suggests LoRA should generalize better than FFT. We tested whether LoRA's constrained updates produce more position-invariant representations. Third, visual feature quality: the SigLIP vision encoder may lack the spatiotemporal understanding that generative video models capture, and fusing frozen video diffusion features (Wan 2.1 [12], via adaptive gated fusion following VEGA-3D [6]) could supply what is missing. We evaluated over 12 configurations of $\pi_{0.5}$ across several LIBERO/LIBERO-PRO suites, with 50 episodes per task.

The first hypothesis is confirmed; the other two are not:

1. **Training recipe (confirmed).** Conservative FFT (batch 64, LR $1e{-}5$) averages 42% on swap, doubling the published 21%, stable from 8 k to 27 k steps. An overly conservative recipe (batch 16, LR $1e{-}6$) drops to 26%.
2. **Finetuning strategy (rejected).** LoRA averages 15 to 21% on swap across 8 k to 30 k steps, consistently below FFT at matched hyperparameters.
3. **Visual feature quality (rejected).** Frozen video diffusion priors reduce swap performance from 42% to 35%. Remaining failures track visual distinctiveness of target objects, suggesting data diversity as the bottleneck.

The contribution is a properly calibrated baseline and the recipe that produces it. Every recent method reporting generalization improvements on LIBERO-PRO evaluates against the published $\pi_{0.5}$ checkpoint. Our results indicate that this baseline is a product of the training recipe, and that reported gains partially compensate for recipe-induced brittleness.

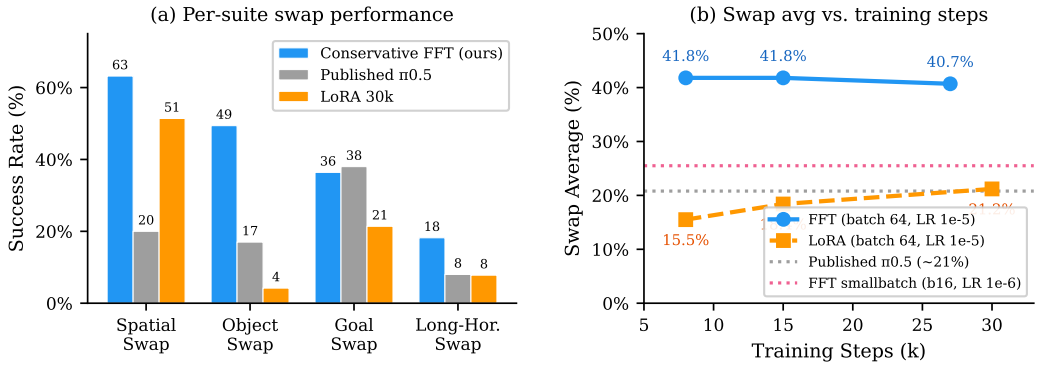

Figure 1: (a) Position-swap success across four LIBERO-PRO suites. Conservative FFT doubles the published checkpoint's swap performance with no architectural modification. (b) Swap average vs. training steps. FFT is stable at ∼42% from 8 k to 27 k; LoRA climbs slowly but never matches FFT. Performance is determined by batch size and learning rate.

2 Related Work

2.1 Finetuning Strategy for VLAs

The field has converged on LoRA [7] as the default for VLA adaptation, with good reason on standard metrics. OpenVLA [8] reports LoRA (rank 32) matches FFT on in-distribution manipulation tasks (68% vs. 70%) at a fraction of the trainable parameters. VLM2VLA [9] represents actions as natural language descriptions, enabling effective LoRA-based adaptation while arguing FFT causes catastrophic forgetting [13] of VLM capabilities. VLA-GSE [10] introduces SVD-based expert routing that outperforms both LoRA and FFT on LIBERO-Plus, and demonstrates that FFT degrades multimodal understanding on benchmarks like MMMU.

These comparisons primarily evaluate on in-distribution or standard generalization metrics. To our knowledge, none test LoRA vs. FFT under perturbation benchmarks that probe visual grounding; LoRA matching FFT on standard LIBERO does not imply it matches under position swap. With small, low-diversity datasets like LIBERO's 50 demonstrations per task, LoRA's concentrated updates ($\sim$32M parameters) may converge faster to dataset shortcuts than FFT's distributed updates ($\sim$2.5B), consistent with Biderman et al.'s [14] observation that LoRA learns less and forgets less.

2.2 Architectural Interventions for VLA Generalization

Several recent methods add components to improve VLA robustness under distribution shift. AFI [4] introduces 3D affordance fields and uses GPT-4o with Grounded-SAM at inference time, reporting improvements on position-displacement perturbations (target object shifted by 5–15 cm from its training position). VLS [5] steers a frozen policy via VLM-based gradient signals at each denoising step, reporting improvements on LIBERO-PRO's position and task perturbations using a LeRobot-finetuned $\pi_{0.5}$ checkpoint. VEGA-3D [6] fuses frozen video diffusion features through adaptive gated attention, reporting 97% on standard LIBERO but not evaluating on LIBERO-PRO.

These methods evaluate against published $\pi_{0.5}$ checkpoints that, as we show, use an overly aggressive training recipe. Reported improvements partially correct for overtraining. We do not claim these methods provide no architectural benefit; we note that the magnitude of their reported gains is measured against a miscalibrated baseline. Notably, AFI's perturbation protocol (position displacement) differs from LIBERO-PRO's (position swap), making direct numerical comparison inappropriate (see Appendix D).

Li et al. [15] offer a competing diagnosis of the same phenomenon, attributing VLA brittleness to spatial-modeling misalignment rather than the finetuning recipe. Our evidence distinguishes the two: changing only the training recipe, without any spatial-modeling intervention, doubles swap performance. This is consistent with the training recipe being a primary lever, though the explanations are not mutually exclusive.

2.3 Perturbation Benchmarks

LIBERO [2] provides four standard suites (spatial, object, goal, long-horizon), each with 10 tasks and 50 demonstrations per task, with fixed object positions across all demonstrations. LIBERO-PRO [3] extends LIBERO with controlled perturbations across several dimensions: position swap, object variant, language, and environment.

Not all perturbation axes are equally diagnostic. Task-level perturbations yield near-zero success for all models tested (under 1% for all configurations in our evaluation). Language perturbations are largely solved: all models in our evaluation score above 90%. Object-variant perturbations change the target object's appearance but not the language instruction, providing a secondary signal. Environment perturbations (lighting, background) were not evaluated because the perturbation BDDL files for this axis were not publicly released. Position swap is the cleanest test of visual grounding of the language instruction: objects exchange locations while the instruction stays the same, and the policy must identify the target by what it looks like rather than where it had memorized the target to

be during training. We focus our analysis on position-swap suites, with object-variant and standard results reported for completeness.

3 Background

3.1 $\pi_{0.5}$ Architecture

$\pi_{0.5}$ [16, 1] combines a PaliGemma [17] vision-language backbone (SigLIP [18] vision encoder with 256 tokens from a 16×16 grid, plus a Gemma 2B language model) with a Gemma 300M action expert that uses flow matching for continuous action prediction. The full model has ∼3.5B parameters.

Under LoRA (rank 32), approximately 32M parameters in the Gemma 2B and Gemma 300M components are trainable. Under full finetuning (FFT), all ∼2.5B parameters in both components are updated. This 78× gap in trainable parameters is one axis of our study; the training recipe (learning rate, batch size, duration) is the other. The interaction between these axes, not either in isolation, determines whether the model retains visual grounding.

3.2 LIBERO and LIBERO-PRO

Standard suites. LIBERO [2] provides 4 suites (spatial, object, goal, libero-10) with 10 tasks each and 50 demonstrations per task. Object positions are fixed across all demonstrations within a task. This design means high scores (>90%) are achievable through trajectory memorization alone: the policy need only reproduce position-conditioned action sequences.

Position-swap suites. LIBERO-PRO [3] extends each standard suite with position-swap variants where objects exchange locations. The instruction stays the same ("pick up the butter"), but the butter is now where the tomato sauce was. A policy relying on position shortcuts will reach for the wrong object. Success requires the model to identify the target by its visual appearance: shape, color, texture, label. This makes position swap the most direct test of whether a policy has learned visual grounding versus trajectory memorization.

LIBERO-PRO also includes object-variant, language, and environment perturbations; we report object-variant results for completeness and discuss other axes in Section 2.3.

4 Experimental Design

4.1 Axes of Investigation

- **Axis 1 — Finetuning strategy:** LoRA (rank 32, ∼32M trainable) vs. full finetuning (∼2.5B trainable), with identical training recipes where possible. *Rationale:* LoRA constrains gradient updates to a low-rank subspace. If visual grounding and trajectory recall compete for limited parameter capacity, LoRA may systematically discard the harder-to-learn signal (appearance) in favor of the simpler one (position).
- **Axis 2 — Training recipe:** Learning rate ($1e{-6}$ to $5e{-5}$), batch size (16–256), and training duration (8 k–30 k steps). *Rationale:* LIBERO provides only 50 demonstrations per task with nearly fixed positions. At the published recipe's batch size of 256 and learning rate of $5e{-5}$, each gradient step aggregates many repeated position-action pairs into large updates that may converge on spatial shortcuts before encoding appearance. We test three regimes: the published recipe (batch 256, LR $5e{-5}$), a conservative recipe (batch 64, LR $1e{-5}$), and an overly conservative recipe (batch 16, LR $1e{-6}$).
- **Axis 3 — Frozen video diffusion features:** Wan 2.1 [12] T2V 1.3B tower (frozen), features from block 20/30, fused via adaptive gated fusion following VEGA-3D [6]. *Rationale:* Video diffusion models trained on internet-scale data encode rich spatial and temporal priors. If the SigLIP vision encoder lacks the spatiotemporal understanding needed

for position-invariant object identification, injecting these features could supply what is missing. Details in Appendix C.

4.2 Configurations

Table 1 summarizes all configurations. Every quantitative claim in Section 5 traces to a row here. The "Swap Avg" column reports mean success across all four position-swap suites, the most diagnostic aggregate metric for visual grounding.

Table 1: Experimental configurations and headline results. Std Avg: mean success across 4 standard LIBERO suites. Swap Avg: mean success across 4 LIBERO-PRO position-swap suites. The published $\pi_{0.5}$ swap numbers are from [3].

Configuration	Strategy	Batch	LR	Steps	Std Avg	Swap Avg
LoRA 8k	LoRA	64	1e-5	8k	86.6%	15.5%
LoRA 15k	LoRA	64	1e-5	15k	90.7%	18.4%
LoRA 30k	LoRA	64	1e-5	30k	94.4%	21.2%
FFT 8k	Full	64	1e-5	8k	96.5%	**41.8%**
FFT 15k	Full	64	1e-5	15k	97.1%	41.8%
FFT 27k	Full	64	1e-5	27k	**97.7%**	40.7%
FFT + WAN w/ GW	Full	64	1e-5	8k	97.0%†	35.3%†
FFT smallbatch	Full	16	1e-6	30k	93.0%†	25.5%†
Published $\pi_{0.5}$	Full	256	5e-5	30k	$\sim$96%	$\sim$21%

†Evaluated with 10 episodes per task; all others use 50.

4.3 Evaluation Protocol

We evaluate on 12 suites: 4 standard LIBERO suites plus 4 position-swap and 4 object-variant LIBERO-PRO suites. Each suite contains 10 tasks evaluated over 50 episodes per task (500 episodes per suite). Success is binary: task completion within the episode horizon. Simulation seeds are fixed for reproducibility.

5 Results

5.1 Standard LIBERO: Conservative FFT Matches Published Performance

Before examining robustness, we verify that the conservative recipe does not sacrifice in-distribution performance. FFT at 8 k steps averages 97% across standard LIBERO suites (spatial 98%, object 99%, goal 97%, libero-10 92%), matching the published $\pi_{0.5}$ checkpoint ($\sim$96%). LoRA at 30 k steps averages 94% (spatial 97%, object 98%, goal 98%, libero-10 84%). Both recipes match or approach the published checkpoint on the metrics the field currently optimizes.

5.2 Position Swap: Batch Size and Learning Rate Determine Visual Grounding

This is the central result. Table 2 reports per-suite performance on position-swap and object-variant perturbations.

Conservative FFT vs. published $\pi_{0.5}$. Conservative FFT (batch 64, LR $1e{-}5$) averages 42% across position-swap suites at 8 k steps, doubling the published checkpoint's 21%. The improvement is concentrated in spatial swap (63% vs. 20%) and object swap (49% vs. 17%), the suites that most directly test whether a policy identifies targets by appearance. On spatial swap, the policy must place an object on a plate that has changed position; on object swap, the target grocery item has moved to a shelf location previously occupied by a different item. Both require the model to understand *what* an object looks like, rather than just reaching for where the object was during training. A striking example from the goal-swap suite: the published checkpoint scores 0% on "turn on the stove" when

Table 2: LIBERO-PRO position-swap results (success rate %). Bold: best in column.

Config	Spatial	Object	Goal	Long Horizon	Avg
LoRA 8k	39	3	17	2	15
LoRA 15k	50	0	19	4	18
LoRA 30k	51	4	21	8	21
FFT 8k	63	49	36	18	**42**
FFT 15k	**65**	47	**37**	18	**42**
FFT 27k	62	**50**	32	**19**	41
FFT + WAN w/ GW 8k	64	32	34	11	35
FFT sm.b. 30k	61	9	23	9	26
Published $\pi_{0.5}$	20	17	38	8	21

nearby objects exchange positions, despite the stove being a fixed appliance whose location does not change. The model has encoded the entire scene layout as a lookup key; perturbing any object, even one irrelevant to the task, invalidates the cached trajectory. Our conservative FFT scores 100% on this task, indicating it has learned to appropriately identify the stove by its visual affordance.

Batch size and learning rate. The same FFT recipe (batch 64, LR $1e{-5}$) averages 42% at 8 k steps, 42% at 15 k, and 41% at 27 k. Swap performance is stable across a $3\times$ range of training duration. The critical variables are batch size and learning rate: the published recipe (batch 256, LR $5e{-5}$) averages 21%, our recipe (batch 64, LR $1e{-5}$) averages 42%, and an overly conservative recipe (batch 16, LR $1e{-6}$) drops to 26%. At batch 256 with a $5\times$ higher learning rate, each gradient step makes large updates driven by many repeated position-action pairs, converging on spatial shortcuts rapidly. At batch 64 with LR $1e{-5}$, updates are smaller and noisier, allowing the model to encode visual features. At batch 16 with LR $1e{-6}$, the gradient signal is too weak: object swap collapses from 49% to 9% while spatial swap stays at 61%, suggesting the model can handle coarse spatial reasoning but cannot learn fine-grained object identity.

LoRA comparison. LoRA averages 16% (8 k), 18% (15 k), and 21% (30 k) on swap, slowly climbing but consistently below FFT at every step count. At matched hyperparameters (batch 64, LR $1e{-5}$), LoRA at 30 k achieves roughly what FFT achieves at any step count on standard LIBERO (94% vs. 97%), but reaches only half the swap performance (21% vs. 42%). LoRA's constrained parameter space ($\sim$32M vs. $\sim$2.5B for FFT) appears unable to encode visual grounding alongside trajectory recall; it prioritizes the simpler function [14].

Comparison with published methods. Evaluation protocols differ across papers, so we report these comparisons as directional rather than definitive (see Appendix D for protocol details). AFI [4] reports improving $\pi_{0.5}$ from 54% to 76% on spatial perturbation using GPT-4o and Grounded-SAM at inference time (albeit with a different position-displacement-based perturbation). VLS [5] reports improving a LeRobot-finetuned $\pi_{0.5}$ checkpoint from 24% to 35% on position perturbation using VLM-based gradient steering, evaluated with 20 episodes per task. Our conservative FFT achieves 42% swap average with the same 50 episodes per task, no inference overhead, and no additional modules. The methods are complementary: AFI and VLS steer a frozen policy at inference time, while our recipe produces a stronger policy to begin with.

5.3 Visual Feature Quality Does Not Explain the Gap

Our third hypothesis was that the SigLIP vision encoder lacks spatiotemporal features that video diffusion models capture, and that fusing frozen Wan 2.1 [12] features (following VEGA-3D [6]) could close the remaining grounding gap. The evidence rejects this. At matched training duration (8 k steps), adding WAN features via adaptive gated fusion reduces average swap performance from 42% to 35%. The largest decline is on object swap: 49% without WAN vs. 32% with WAN, a

17-point degradation. Spatial swap is unchanged (63% vs. 64%). On object-variant suites, WAN provides a mixed signal: slight improvements on some suites, slight declines on others.

Naively adding a learnable gate between SigLIP and WAN features results in the gate learning to ignore WAN entirely (converging to ∼97% SigLIP, 3% WAN), because it is easier for gradient descent to discard unfamiliar features than to learn to use them. We addressed this with a tent-shaped gate warmup (GW) schedule that forces WAN utilization up to 80% before gradually freeing the gate. This shifted convergence to ∼75% SigLIP and 25% WAN, confirming the model was using WAN features. Even with meaningful WAN utilization, swap performance did not improve.

With LoRA, WAN also provides no benefit: DeepLoRA with gate warmup (20% swap avg) is indistinguishable from the LoRA baseline (16–21%).

This negative result is itself informative [19, 20]. When the training data never varies object positions, richer visual features cannot teach position-invariant identification because there is no gradient signal for that behavior.

5.4 Per-Task Analysis: Visual Distinctiveness Predicts Success

The aggregate 42% swap average conceals a bimodal distribution. Figure 2 decomposes object-swap results by task, labeling each bar with the actual swap pairing from the LIBERO-PRO BDDL files. Success tracks visual distinctiveness: cream cheese ↔ alphabet soup (box vs. can) achieves 100%, while ketchup ↔ cream cheese and milk ↔ cream cheese both yield 0%, because cream cheese occupies the target's memorized location and the policy grabs whatever sits there. The same pattern holds across all four swap suites (see Appendix B for per-suite breakdowns): plate ↔ cookie box succeeds on spatial swap (94–100%), while plate ↔ black bowl fails (0–90%); "turn on the stove" achieves 100% after swap because the stove has a unique affordance (knob pressing); book ↔ mug achieves 90% on libero-10 (maximally distinct shapes).

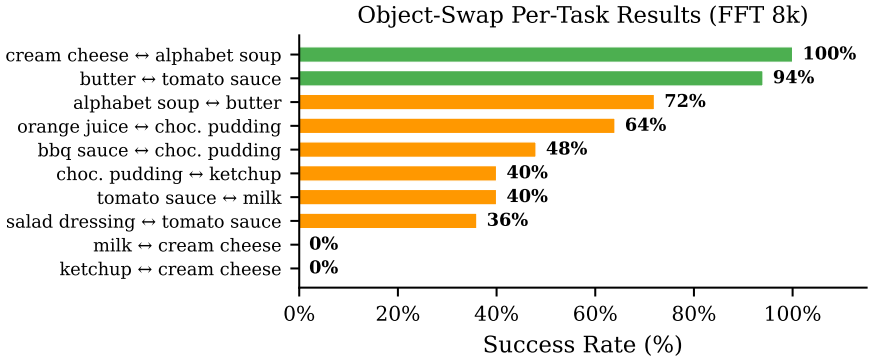

Figure 2: Per-task object-swap success for FFT 8 k. Each bar is labeled with the swap pairing. Visually distinct pairs succeed.

Three factors predict per-task swap success: (1) visual distinctiveness of the swapped pair (dominant), (2) functional uniqueness of the target, and (3) severity of scene layout disruption. These patterns are consistent across FFT, LoRA, and WAN configurations, confirming they reflect properties of the training data rather than model-specific memorization [21].

6 Discussion

6.1 Why the Conservative Recipe Works

Two factors interact. LoRA's ∼32M trainable parameters force trajectory recall (the simpler function) to dominate over visual grounding; FFT's ∼2.5B parameters allow both pathways to coexist [22]. Given sufficient parameters, gradient dynamics determine whether shortcuts dominate: at

batch 256 and LR $5e{-5}$, large updates converge rapidly on spatial shortcuts [21]; at batch 64 and LR $1e{-5}$, noisier gradients allow visual features to be encoded before the simpler mapping takes over. The stability across 8 k to 27 k steps suggests that the right batch size and learning rate are not hypersensitive to step size.

Interestingly, validation loss did not predict swap performance. Val loss reaches its minimum around 5 k to 8 k steps for both LoRA and FFT, then rises, yet standard LIBERO performance keeps improving (LoRA) or plateaus (FFT). Swap performance follows neither curve. This confirms that validation loss measures fidelity to the training distribution, not generalization under perturbation [23].

6.2 The Data Diversity Bottleneck

Conservative FFT addresses recipe-induced memorization but cannot overcome data-induced ambiguity. When target and distractor objects are visually similar, no recipe can teach the model to distinguish them, because the training data contains no examples of the distinction mattering. LIBERO-Plus [24] confirms that data diversity is the key lever: training on 20 k+ diversified trajectories raised overall perturbation performance to 80%. Combining calibrated recipes with position-diverse training data is the natural next step.

6.3 Implications for Baseline Evaluation

Methods reporting generalization improvements on LIBERO-PRO evaluate against the published $\pi_{0.5}$ checkpoint, which uses an overly aggressive training recipe. Reported gains partially reflect recovery from recipe-induced brittleness. A method that improves over our conservative FFT baseline demonstrates architectural benefit independent of training recipe; a method that only improves over the published checkpoint may be correcting what a recipe change would have corrected at zero cost. We release our configurations and per-task results as a calibrated baseline for future work.

7 Limitations

1. **Simulation only.** All experiments run in LIBERO simulation. Sim-to-real transfer of the robustness findings is untested.
2. **Single VLA backbone.** We evaluate only $\pi_{0.5}$. The interaction between batch size, learning rate, and visual grounding may manifest differently on other architectures (OpenVLA, RT-2, Octo).
3. **Partial grounding.** Conservative FFT averages 42% on swap, not 90%+. As Section 5.4 shows, the remaining failures suggest training data diversity as the bottleneck: 50 demonstrations with fixed positions provide no signal for distinguishing visually similar objects. Closing this gap requires data-level interventions.

8 Conclusion

We investigated VLA trajectory memorization on LIBERO-PRO and found that the training recipe is a major lever in the generalization problem. A conservative full-finetuning recipe (batch size 64, learning rate $1e{-5}$) doubles the published $\pi_{0.5}$ checkpoint's position-swap performance (42% vs. 21%) while matching its standard LIBERO scores, with no architectural changes. This improvement is stable across training durations from 8 k to 27 k steps; the critical variables are batch size and learning rate, not when training stops. LoRA finetuning cannot achieve comparable swap performance at any training duration, and frozen video diffusion priors provide no benefit.

The published $\pi_{0.5}$ checkpoint, which serves as the baseline for recent generalization methods, is a product of its training recipe. We provide a calibrated alternative and release our configurations and full per-task results. The remaining bottleneck is training data diversity: 50 demonstrations with fixed object positions cannot teach position-invariant identification of visually similar objects. Combining calibrated finetuning recipes with position-diverse training data is the natural next step.

References

[1] Physical Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke, A. Walling, H. Wang, L. Yu, and U. Zhilinsky. $\pi_{0.5}$: A vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.

[2] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36, 2023.

[3] X. Zhou, Y. Xu, G. Tie, Y. Chen, G. Zhang, D. Chu, P. Zhou, and L. Sun. LIBERO-PRO: Towards robust and fair evaluation of vision-language-action models beyond memorization. *arXiv preprint arXiv:2510.03827*, 2025.

[4] S. Xu, Z. Wang, Y. Wang, C. Xia, T. Huang, and C. Xu. Affordance field intervention: Enabling VLAs to escape memory traps in robotic manipulation. *arXiv preprint arXiv:2512.07472*, 2025.

[5] S. Liu, I. S. Singh, Y. Xu, J. Duan, and R. Krishna. VLS: Steering pretrained robot policies via vision-language models. *arXiv preprint arXiv:2602.03973*, 2026.

[6] X. Wu, D. Liang, T. Feng, K. Xia, Y. Zhang, X. Li, X. Tan, and X. Bai. Generation models know space: Unleashing implicit 3D priors for scene understanding. *arXiv preprint arXiv:2603.19235*, 2026.

[7] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[8] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning (CoRL)*, 2024.

[9] A. J. Hancock, X. Wu, L. Zha, O. Russakovsky, and A. Majumdar. Actions as language: Fine-tuning VLMs into VLAs without catastrophic forgetting. *arXiv preprint arXiv:2509.22195*, 2025.

[10] Y. Jiang, J. Lu, X. Qin, X. Chen, K. Wang, F. Gao, and L. Zhao. VLA-GSE: Boosting parameter-efficient fine-tuning in VLA with generalized and specialized experts. *arXiv preprint arXiv:2605.06175*, 2026.

[11] M. J. Kim, C. Finn, and P. Liang. Fine-tuning vision-language-action models: Optimizing speed and success. In *Robotics: Science and Systems*, 2025.

[12] Team Wan et al. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025.

[13] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

[14] D. Biderman, J. Portes, J. J. Gonzalez Ortiz, M. Paul, P. Greengard, C. Jennings, D. King, S. Havens, V. Chiley, J. Frankle, C. Blakeney, and J. P. Cunningham. LoRA learns less and forgets less. *Transactions on Machine Learning Research*, 2024.

[15] W. Li, Q. Zhang, R. Zhai, L. Lin, and G. Wang. VLA models are more generalizable than you think: Revisiting physical and spatial modeling. *arXiv preprint arXiv:2512.02902*, 2025.

[16] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. π_0: A vision-language-action flow model for general robot control. In *Robotics: Science and Systems*, 2025.

[17] L. Beyer, A. Steiner, A. S. Pinto, A. Kolesnikov, X. Wang, D. Salz, M. Neumann, I. Alabdulmohsin, M. Tschannen, E. Bugliarello, et al. PaliGemma: A versatile 3b VLM for transfer. *arXiv preprint arXiv:2407.07726*, 2024.

[18] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer. Sigmoid loss for language image pre-training. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

[19] K. Musgrave, S. Belongie, and S.-N. Lim. A metric learning reality check. In *European Conference on Computer Vision*, 2020.

[20] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger. Deep reinforcement learning that matters. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), 2018.

[21] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020.

[22] A. Aghajanyan, S. Gupta, and L. Zettlemoyer. Intrinsic dimensionality explains the effectiveness of language model fine-tuning. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021.

[23] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals. Understanding deep learning requires rethinking generalization. In *International Conference on Learning Representations*, 2017.

[24] S. Fei, S. Wang, J. Shi, Z. Dai, J. Cai, P. Qian, L. Ji, X. He, S. Zhang, Z. Fei, J. Fu, J. Gong, and X. Qiu. LIBERO-Plus: In-depth robustness analysis of vision-language-action models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026.

A Training and Hyperparameter Details

All configurations start from the same pretrained $\pi_{0.5}$ base checkpoint (`pi05_base` from the `openpi` repository). Training uses AdamW with gradient clipping at norm 1.0. The learning rate follows a cosine decay schedule in all cases. Table 3 reports the full hyperparameter set for each configuration family.

Table 3: Training hyperparameters for all configuration families.

	Published $\pi_{0.5}$	Our FFT	FFT Smallbatch	Our LoRA
Trainable params	∼2.5B	∼2.5B	∼2.5B	∼32M
Batch size	256	64	16	64
Peak learning rate	5e-5	1e-5	1e-6	1e-5
Decay learning rate	—	1e-6	1e-6	1e-6
Warmup steps	10,000	1,000	1,000	1,000
Decay steps	—	30,000	30,000	30,000
Max training steps	30,000	30,000	30,000	30,000
EMA decay	0.999	0.999	0.999	None
Action horizon	10	10	10	10
LoRA rank	—	—	—	32
Optimizer	—	AdamW	AdamW	AdamW
Gradient clip norm	—	1.0	1.0	1.0

The published $\pi_{0.5}$ hyperparameters are taken from the `openpi` repository's `pi05_libero` configuration. We do not have access to the exact optimizer or decay schedule used for the published checkpoint; the values shown are from the repository default. Checkpoints were saved at 4 k, 8 k, 15 k, and 27–30 k steps for evaluation. All configurations use the same training data: the `physical-intelligence/libero` dataset (50 demonstrations per task across 4 suites, 40 tasks total after holding out a validation split).

For WAN fusion configurations, the frozen Wan 2.1 T2V 1.3B tower is never updated. Only the gated fusion layers and (for FFT) the full $\pi_{0.5}$ parameters are trained. See Appendix C for fusion-specific details.

B Per-Task Swap Breakdowns

Section 5.4 summarizes the per-task pattern; here we provide the full breakdowns.

On **spatial swap** (target: place bowl on plate), performance depends on what the plate swaps with: cookie box (94–100%), another black bowl (0–90%), ramekin (2–60%). On **object swap** (target: pick up grocery item), cream cheese ↔ alphabet soup achieves 100%, butter ↔ tomato sauce 94%, while ketchup ↔ cream cheese and milk ↔ cream cheese both yield 0%. On **goal swap**, "turn on the stove" achieves 100% (unique affordance), while wine rack ↔ wooden cabinet yields 0% (visually identical furniture). "Open drawer, put bowl inside" achieves 90% (cream cheese ↔ bowl, distinct shapes). On **libero-10 swap**, "pick up book, place in caddy" achieves 90% (book ↔ mug, maximally distinct), while multi-step grocery tasks with similar-looking item swaps yield 0–4%. "Put cream cheese and butter in basket" achieves 52% (cream cheese and butter are moderately distinct from their swap partners).

C Adaptive Gated Fusion

We follow VEGA-3D [6] (Eqs. 6–8) for fusing frozen video diffusion features with the SigLIP visual tokens. This appendix documents the implementation since it is not the main contribution.

Feature extraction. We use the Wan 2.1 Text-to-Video 1.3B model (1536-dim residual stream, 30 transformer blocks, patch size $(1, 2, 2)$). The tower is always frozen. For each camera image, the

input is normalized to $[-1, 1]$, letterbox-padded to the WAN resolution, encoded through the VAE, and noised at a fixed diffusion timestep $\tau = 300$. The noisy latent is passed through the frozen DiT, and a forward hook on block 20 (of 30) captures intermediate features. These are reshaped to a spatial grid and average-pooled to $16 \times 16 = 256$ tokens to match SigLIP's token count, yielding $F_{\text{gen}} \in \mathbb{R}^{B\times 256\times 1536}$. Features are precomputed offline and cached as per-episode safetensors files to avoid instantiating the tower during training.

Gate equations. Let F_{gen} denote the projected WAN features and F_{sem} the (optionally projected) SigLIP tokens, both in $\mathbb{R}^{B\times 256\times D}$ where $D = 2048$ (Gemma-2B width). The projection $P_{\text{gen}} : \mathbb{R}^{1536} \to \mathbb{R}^{2048}$ is a learned linear layer. An optional $P_{\text{sem}} : \mathbb{R}^{2048} \to \mathbb{R}^{2048}$ projects the SigLIP stream (identity-initialized in some configurations).

The per-token gate is:

$$g_i = \sigma\big(W_g \cdot [\text{LN}(F_{\text{gen},i})\,;\, \text{LN}(F_{\text{sem},i})] + b_g\big) \tag{1}$$

where LN is LayerNorm applied independently to each stream, $[\cdot\,;\,\cdot]$ is concatenation, and $W_g \in \mathbb{R}^{1\times 2D}$ with bias b_g are learned. The fused output is a convex combination:

$$F_{\text{fused},i} = (1 - g_i) \cdot F_{\text{gen},i} + g_i \cdot F_{\text{sem},i} \tag{2}$$

The convex form prevents signal amplification that would destabilize downstream attention layers. When $g_i = 1$, the output is pure SigLIP; when $g_i = 0$, pure WAN.

Tent-shaped gate warmup. Without intervention, the gate converges to $g \approx 0.97$ (nearly pure SigLIP), because discarding unfamiliar WAN features is easier than learning to use them. To force meaningful WAN utilization, we apply a two-phase cosine warmup schedule over W steps with peak at $W/2$:

Phase 1 (step 0 to $W/2$): override the learned gate with a forced value that cosine-anneals from g_{start} (default 1.0, pure SigLIP) to g_{target} (default 0.2, 80% WAN):

$$g_{\text{forced}} = g_{\text{start}} + (g_{\text{target}} - g_{\text{start}}) \cdot \tfrac{1}{2}(1 - \cos(\pi \cdot t_1)) \tag{3}$$

where $t_1 = \text{clip}(\text{step}/(W/2),\, 0,\, 1)$.

Phase 2 ($W/2$ to W): cosine-anneal from the forced target back to the fully learned gate:

$$g = \lambda \cdot g_{\text{target}} + (1 - \lambda) \cdot g_{\text{learned}}, \quad \lambda = \tfrac{1}{2}(1 + \cos(\pi \cdot t_2)) \tag{4}$$

where $t_2 = \text{clip}((\text{step} - W/2)/(W/2),\, 0,\, 1)$.

After step W, the gate is fully learned. In our FFT + WAN w/ GW configuration, $W = 4000$ and $g_{\text{target}} = 0.2$. After warmup, the gate converges to $g \approx 0.75$ (75% SigLIP, 25% WAN), confirming meaningful WAN utilization. Even with this utilization, swap performance did not improve over the FFT baseline without WAN (Section 5.3).

Trainable parameters. The fusion adds a small number of trainable parameters: P_{gen} (1536×2048), P_{sem} (2048×2048, optional), two LayerNorms (2×2048), and W_g (1×4096). These are trained alongside the main model parameters (LoRA adapters or full FFT). The frozen WAN tower ($\sim$1.3B parameters) is not loaded during training when features are precomputed.

D Comparison Protocol Notes

Numerical comparisons with AFI [4] and VLS [5] in the main text are directional, not definitive. Table 4 summarizes the key protocol differences.

AFI protocol differences. AFI [4] describes its simulation evaluation as "spatial perturbations to target object positions ... following the OOD evaluation protocol from LIBERO-Pro." However,

Table 4: Evaluation protocol comparison across studies.

	Ours	AFI	VLS
Perturbation type	Position swap (objects exchange locations)	Position displacement (target shifted 5–15 cm)	Position swap (LIBERO-PRO protocol)
$\pi_{0.5}$ checkpoint	openpi official	openpi official	LeRobot community
Episodes per task	50	Not specified	20
Suites evaluated	All 4 swap suites	Spatial, Object only (7/10 Spatial tasks)	All 4 suites
Includes task perturb.	Reported separately	Not evaluated	Included in "overall"
Inference additions	None	GPT-4o + Grounded-SAM + depth estimation	VLM gradient steering + FK resampling

their baseline $\pi_{0.5}$ numbers (54.0% on LIBERO-Spatial, 56.4% on LIBERO-Object) differ substantially from LIBERO-PRO's published numbers for the same checkpoint (20% and 17% respectively). Per-task comparison confirms the discrepancy: for example, on "Pick(table_center, plate)," AFI reports 96% while LIBERO-PRO reports 0%. AFI's Figure 3 caption describes "object position perturbations ... displaced to significantly deviated positions," and their real-world protocol specifies 5–15 cm displacement. We conclude that AFI evaluates position displacement (the target object is moved to a nearby location) rather than position swap (two objects exchange locations). Displacement is a substantially easier test because no confuser object occupies the target's original position. AFI also evaluates only 7 of 10 LIBERO-Spatial tasks and does not specify episode counts for simulation.

VLS protocol differences. VLS [5] evaluates on LIBERO-PRO's position and task perturbations using 20 episodes per task (vs. our 50). Their baseline is a LeRobot-finetuned $\pi_{0.5}$ checkpoint (`lerobot/pi05_libero_finetuned` on HuggingFace), not the standard openpi checkpoint. This LeRobot checkpoint shows markedly different baseline performance from the openpi checkpoint: 23.1% on task perturbation (vs. 0.75% for openpi $\pi_{0.5}$) and 24.3% on position perturbation (vs. 20.8% for openpi). The training recipe differences between the LeRobot and openpi checkpoints are not documented in the VLS paper. VLS's reported "overall" metric (36.8%) averages across both task and position perturbation columns; their position-swap-specific average is 35.1%.

Implications. These protocol differences mean that AFI's "54% $\rightarrow$ 76%" and VLS's "24% $\rightarrow$ 37%" cannot be directly compared to our "21% $\rightarrow$ 42%." The perturbation types, baselines, and episode counts all differ. We report these comparisons in the main text as evidence that a recipe change with zero inference cost achieves competitive robustness, not as claims of strict superiority.